Groupe 18

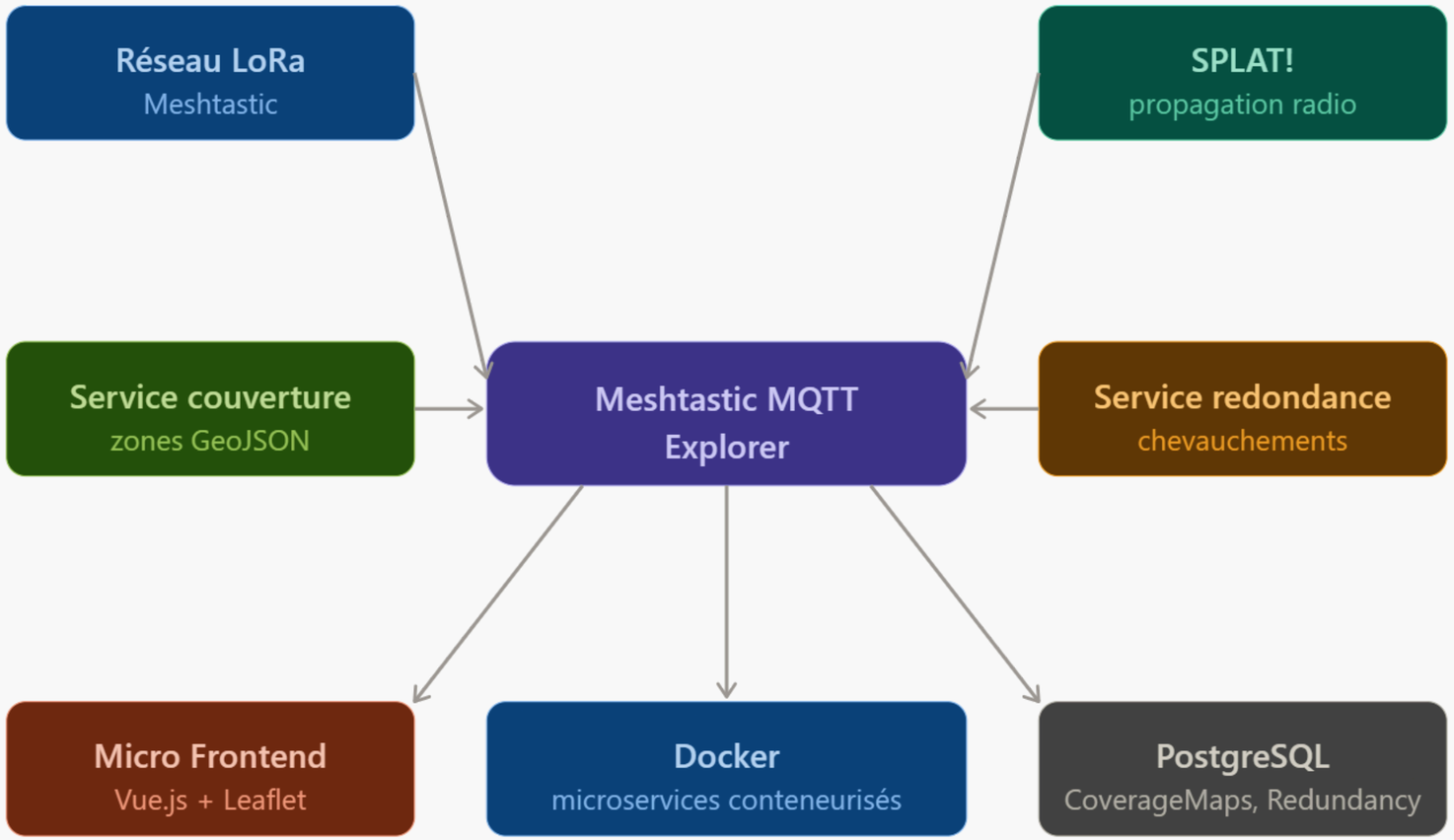
Micro-Service et Micro-Frontend pour SPLAT! (RF Signal Propagation, Loss, And Terrain)

2025/2026

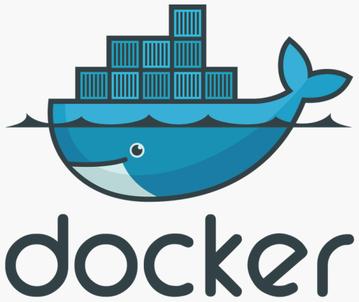
Encadré par :
DONSEZ didier

Réalisé par :
ELOUAZZANI Soufian
BOUTI Chaimae

Contexte et projet à réaliser

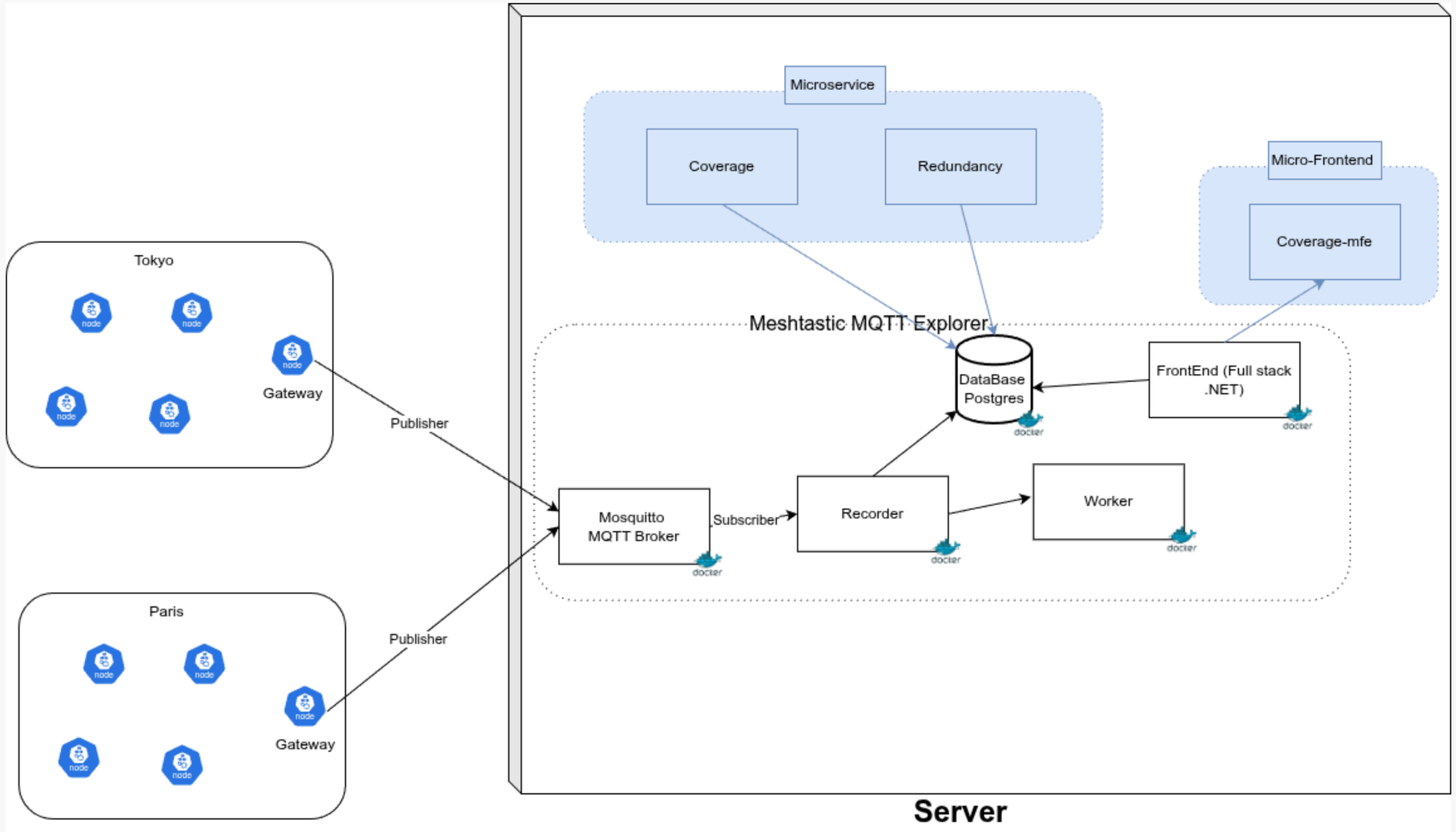


Technologies utilisés



Intégration avec l'existant

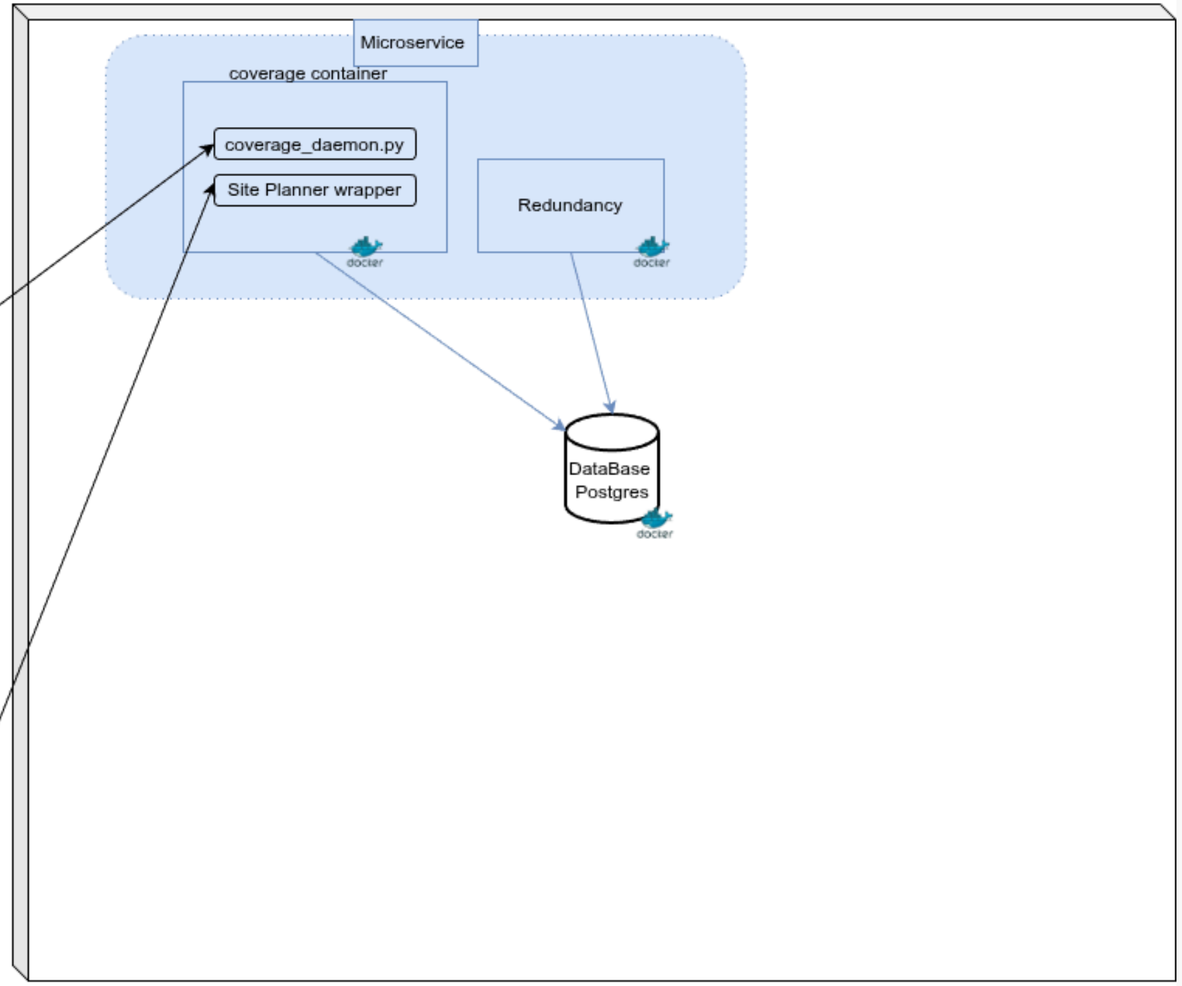
Architecture proposée :



Service de couverture

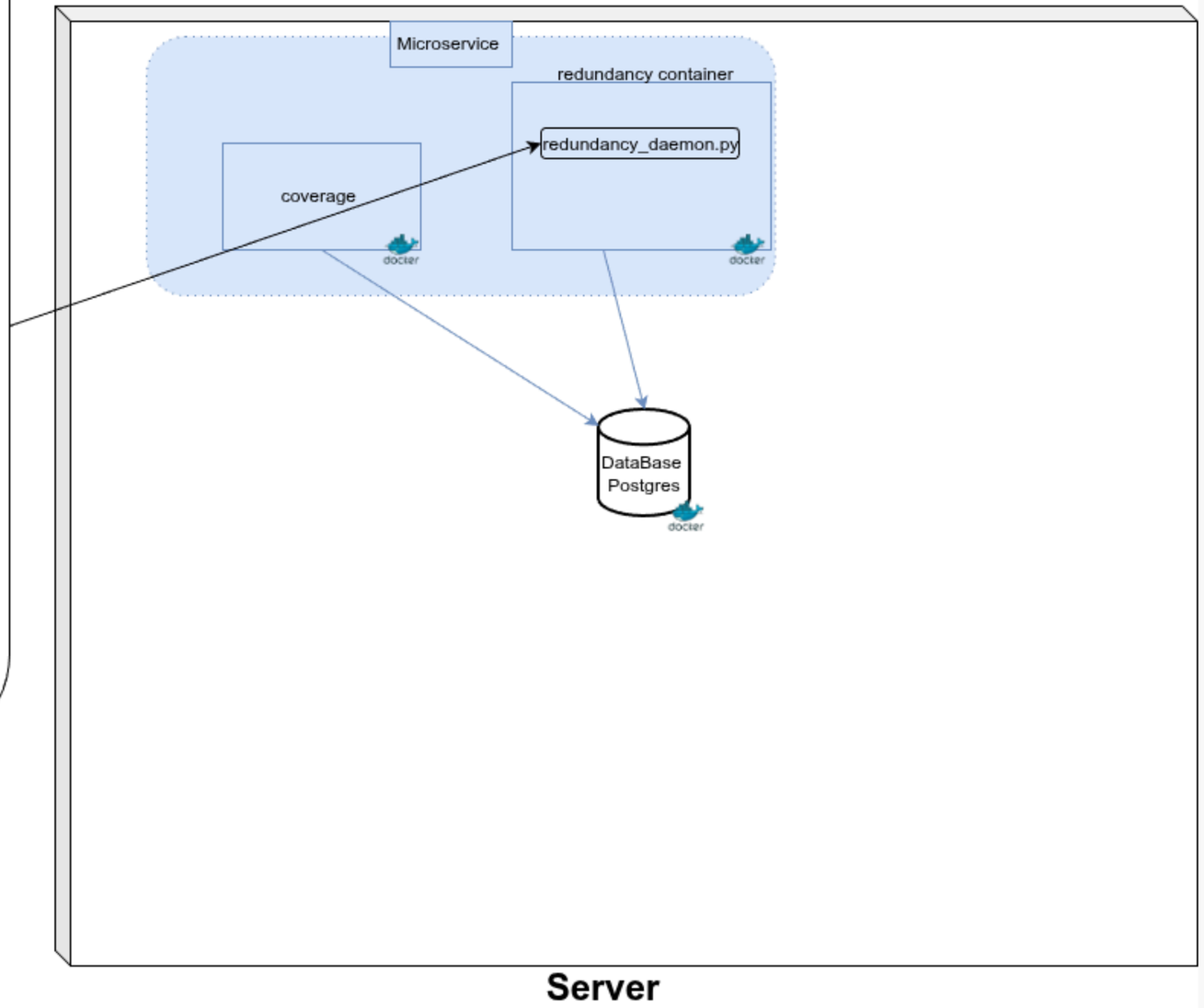
```
while True:

    # 1. Connect to database
    conn = get_db_connection()


    # 2. Get all nodes with GPS coordinates
    nodes = get_active_nodes(conn)


    # 3. For each node, calculate coverage
    for node in nodes:
        if needs_recalculation(conn, node.id):
            # Call SPLAT! via Site Planner wrapper
            geojson = calculate_node_coverage(node)


            # Save to database
            save_coverage_map(conn, node.id, geojson)


    # 4. Wait X seconds before next run (default 300s or 24h)
    time.sleep(RUN_INTERVAL)
```

```
#Site Planner wrapper
app/services/splat.py

#SPLAT! binary
splat/splat
```

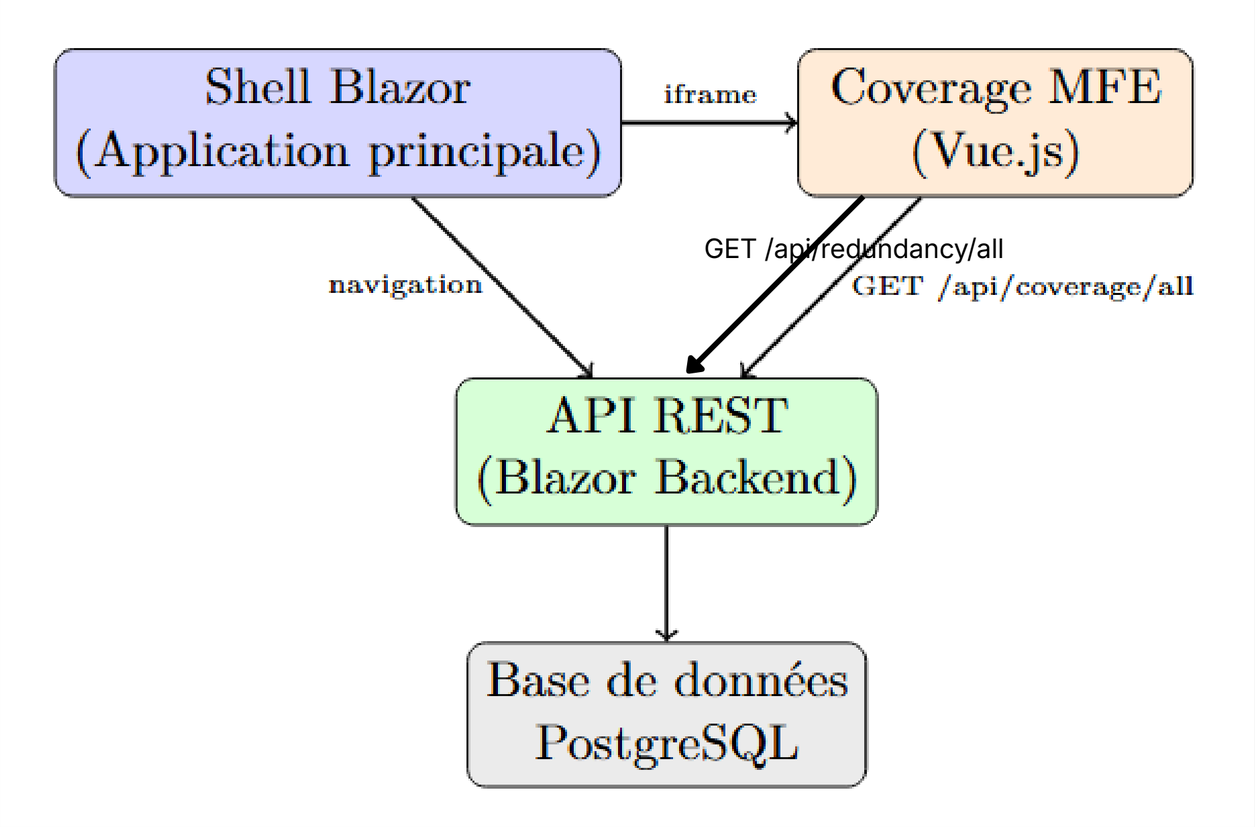


Service de redondance

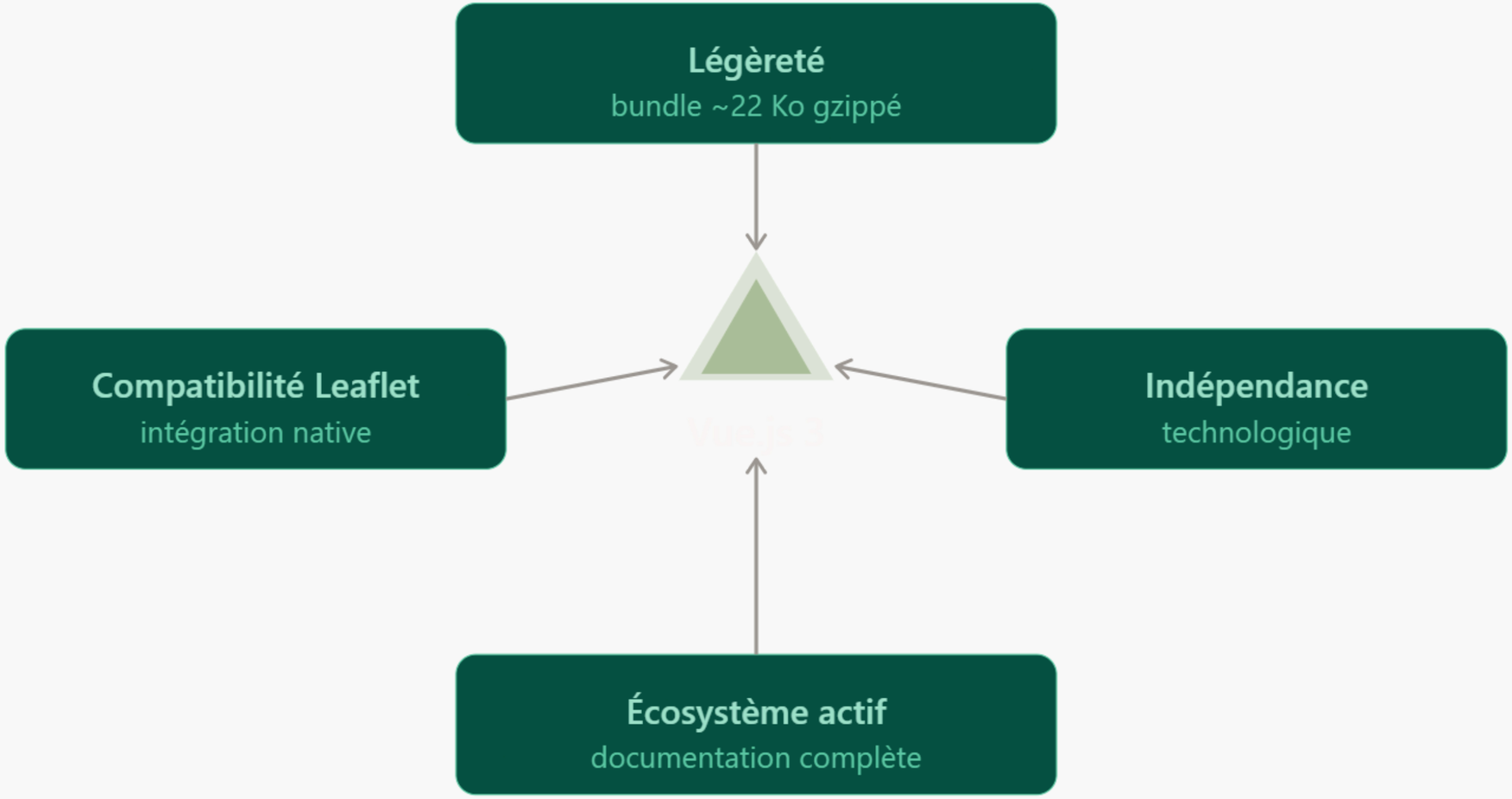
```
def calculate_overlap(node1, node2):

    # Get centers and radii from coverage polygons
    lat1, lon1, r1 = get_circle(node1)
    lat2, lon2, r2 = get_circle(node2)

    # Calculate distance between centers
    distance = sqrt((lat1-lat2)**2 + (lon1-lon2)**2)

    # Check if circles overlap
    if distance < (r1 + r2):
        return True, overlap_score
    return False, 0

def get_redundancy_level(overlap_count):
    if overlap_count >= 5: return 5, '#ff0000'  # Red
    if overlap_count >= 4: return 4, '#ff6600'  # Orange-red
    if overlap_count >= 3: return 3, '#ffa500'  # Orange
    if overlap_count >= 2: return 2, '#ffff00'  # Yellow
    return 1, '#00ff00'  # Green
```



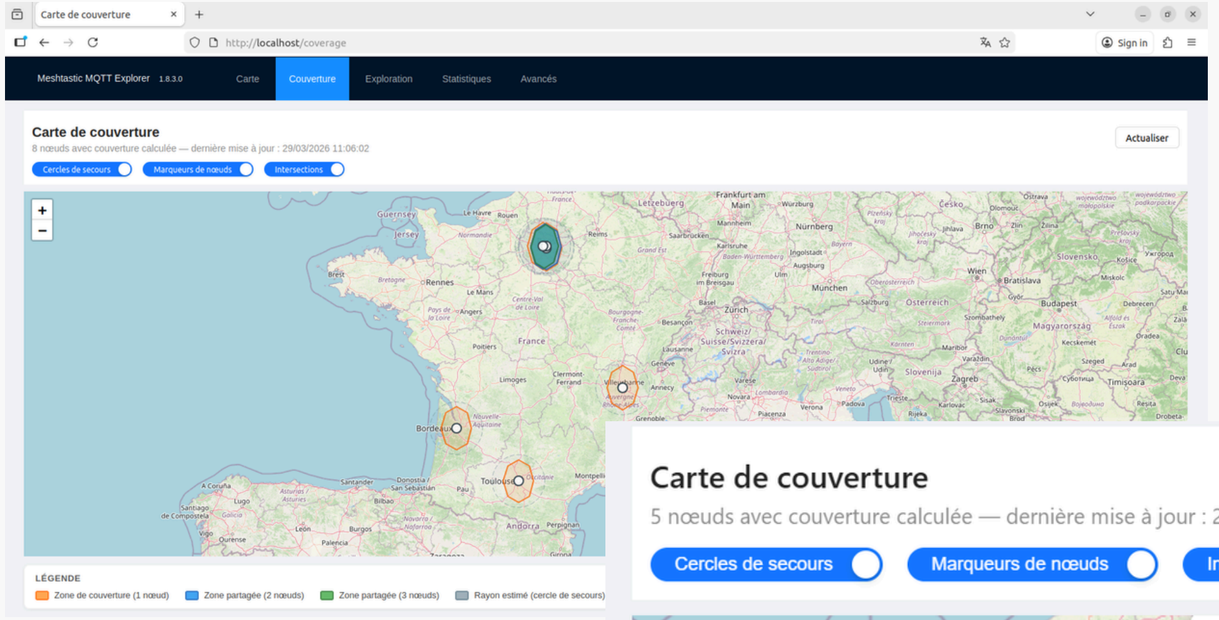
Micro-frontend de couverture et de redandence

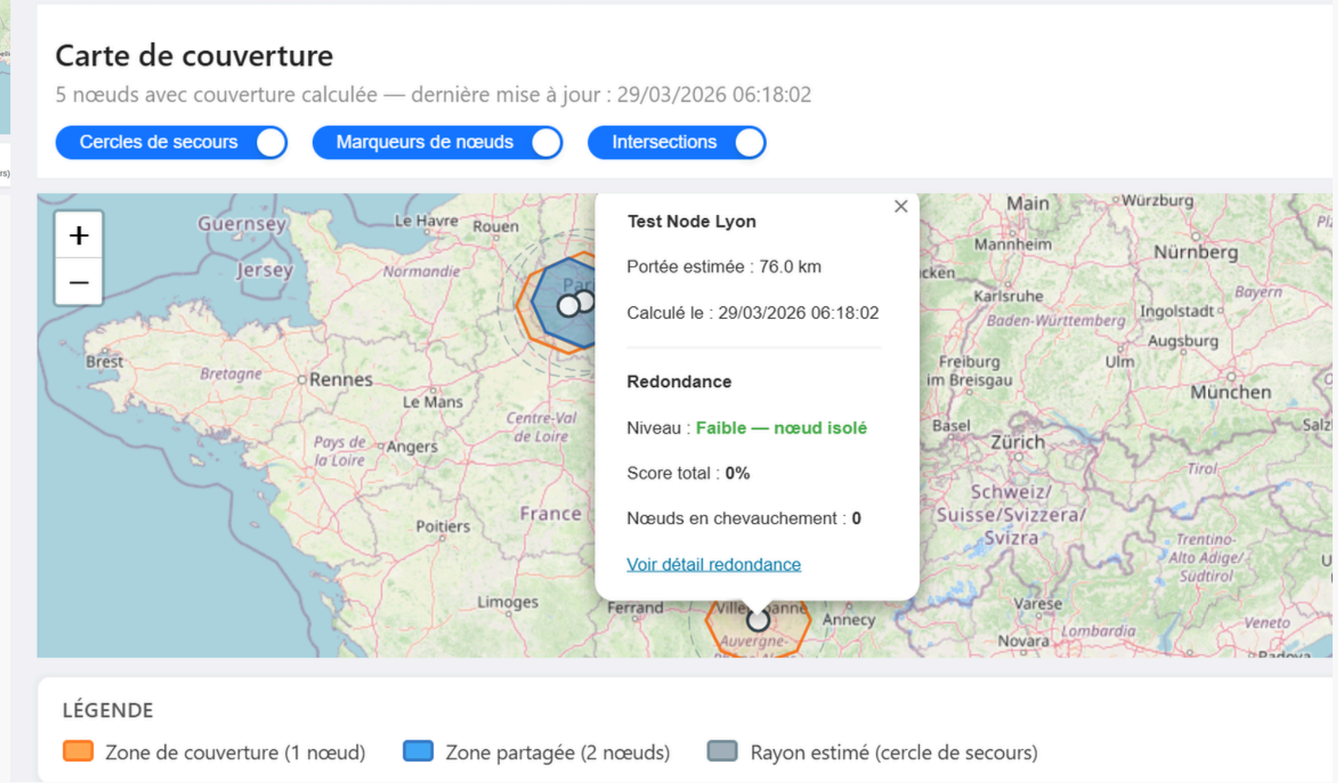


Choix de Vue.js

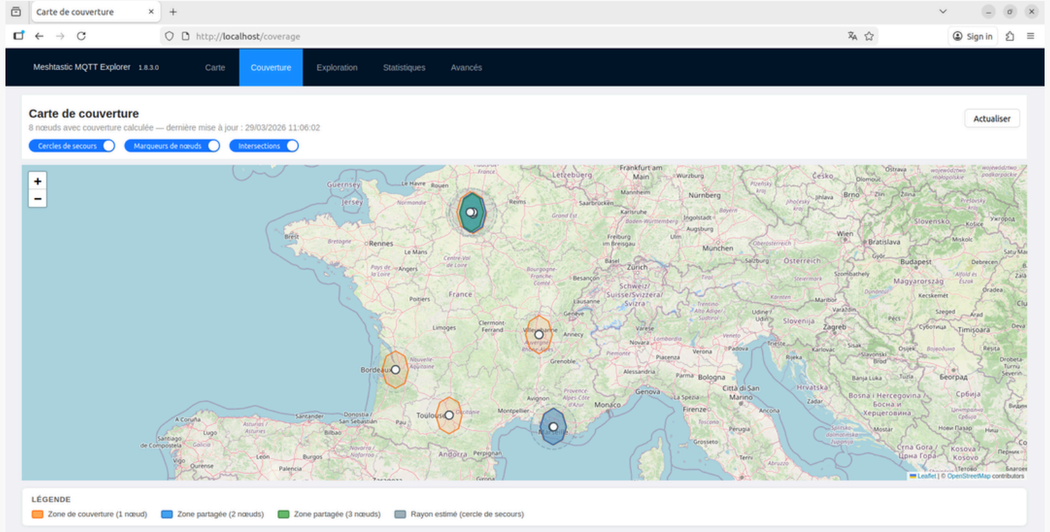


Résultats


Carte de couverture
http://localhost/coverage
Sign in
Meshtastic MQTT Explorer 1.8.3.0
Carte
Couverture
Exploration
Statistiques
Avancés
Carte de couverture
8 nœuds avec couverture calculée — dernière mise à jour : 29/03/2026 11:06:02
Actualiser
Cercles de secours
Marqueurs de nœuds
Intersections
LÉGENDE
Zone de couverture (1 nœud)
Zone partagée (2 nœuds)
Zone partagée (3 nœuds)
Rayon estimé (cercle de secours)


Carte de couverture
5 nœuds avec couverture calculée — dernière mise à jour : 29/03/2026 06:18:02
Cercles de secours
Marqueurs de nœuds
Intersections
Test Node Lyon
Portée estimée : 76.0 km
Calculé le : 29/03/2026 06:18:02
Redondance
Niveau : Faible — nœud isolé
Score total : 0%
Nœuds en chevauchement : 0
Voir détail redondance
LÉGENDE
Zone de couverture (1 nœud)
Zone partagée (2 nœuds)
Rayon estimé (cercle de secours)

Résultats




```
postgres=# \dt
                   List of tables
 Schema |          Name           | Type  |  Owner
--------+-------------------------+-------+----------
 public | Channels                | table | postgres
 public | CoverageMaps            | table | postgres
 public | MqttServers             | table | postgres
 public | NeighborInfos           | table | postgres
 public | Nodes                   | table | postgres
 public | Packets                 | table | postgres
 public | PaxCounters             | table | postgres
 public | Positions               | table | postgres
 public | Redundancy              | table | postgres
 public | RedundancyMap           | table | postgres
 public | SignalHistories         | table | postgres
 public | Telemetries             | table | postgres
 public | TextMessages            | table | postgres
 public | Waypoints               | table | postgres
 public | Webhooks                | table | postgres
 public | WebhooksHistories       | table | postgres
 public | __EFMigrationsHistory   | table | postgres
(17 rows)

postgres=# SELECT "Id", "LongName", "Latitude", "Longitude" FROM "Nodes" LIMIT 10;
   Id    |        LongName        | Latitude | Longitude
--------+-----------------------+----------+-----------
 111111 | Paris Central         |  48.8566 |    2.3522
 111112 | Paris Montmartre      |  48.8867 |    2.3431
 111113 | Paris La Defense      |  48.8926 |     2.238
 222221 | Lyon Centre           |   45.764 |    4.8357
 333331 | Marseille Centre      |  43.2965 |    5.3698
 333332 | Marseille Vieux Port  |  43.2962 |    5.3752
 444441 | Toulouse Centre       |  43.6047 |    1.4442
 555551 | Bordeaux Centre       |  44.8378 |   -0.5792
```

```
stgres=# SELECT "NodeId", "Data"->>'nodeName' as Node, "Data"->>'redundancyLevel' as Level FROM "Redundancy";
odeId |         node          | level
------+-----------------------+-------
11113 | Paris La Defense      | 3
22221 | Lyon Centre           | 1
33331 | Marseille Centre      | 2
33332 | Marseille Vieux Port  | 2
44441 | Toulouse Centre       | 1
55551 | Bordeaux Centre       | 1
11111 | Paris Central         | 3
11112 | Paris Montmartre      | 3
 rows)
```

```
postgres=# SELECT "Level", "Color", "NodeIds" FROM "RedundancyMap" ORDER BY "Level";
 Level |  Color  |          NodeIds
-------+---------+--------------------------
     1 | #00ff00 | [222221, 444441, 555551]
     2 | #ffff00 | [333331, 333332]
     3 | #ffa500 | [111111, 111112, 111113]
(3 rows)
```

```
 NodeId |         CalculatedAt          |                                                
```

Merci